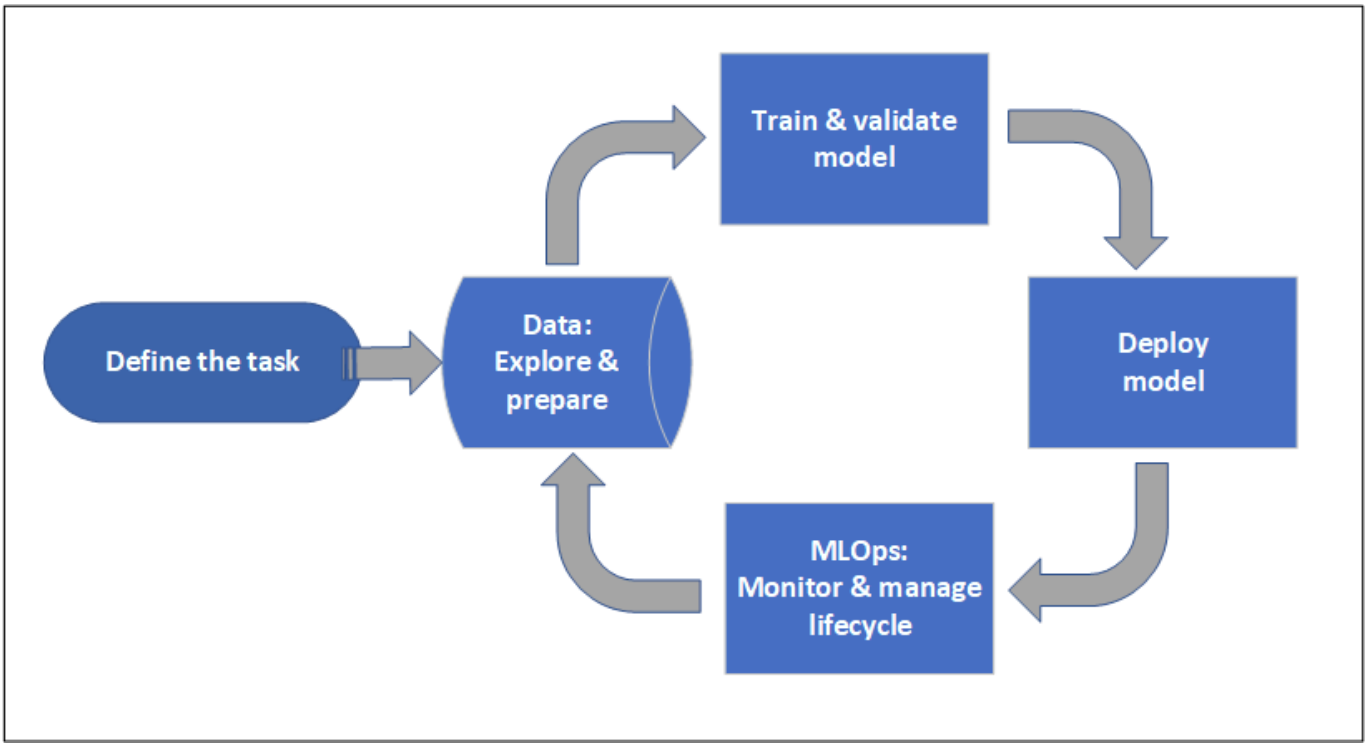
Azure Machine Learning 调研

0. 平台入口

项目	内容
产品名	Azure Machine Learning（Azure ML，也写作 Azure AI Machine Learning Studio）
控制台	https://ml.azure.com
文档	https://learn.microsoft.com/azure/machine-learning/
架构与核心概念	https://learn.microsoft.com/azure/machine-learning/concept-azure-machine-learning-architecture
无码训练（对标百度 BML 预置调参）	Automated ML / Designer / Model catalog
计费模型	按计算资源时长 + 存储 + 端点实例计费；训练集群可缩容到 0 节点

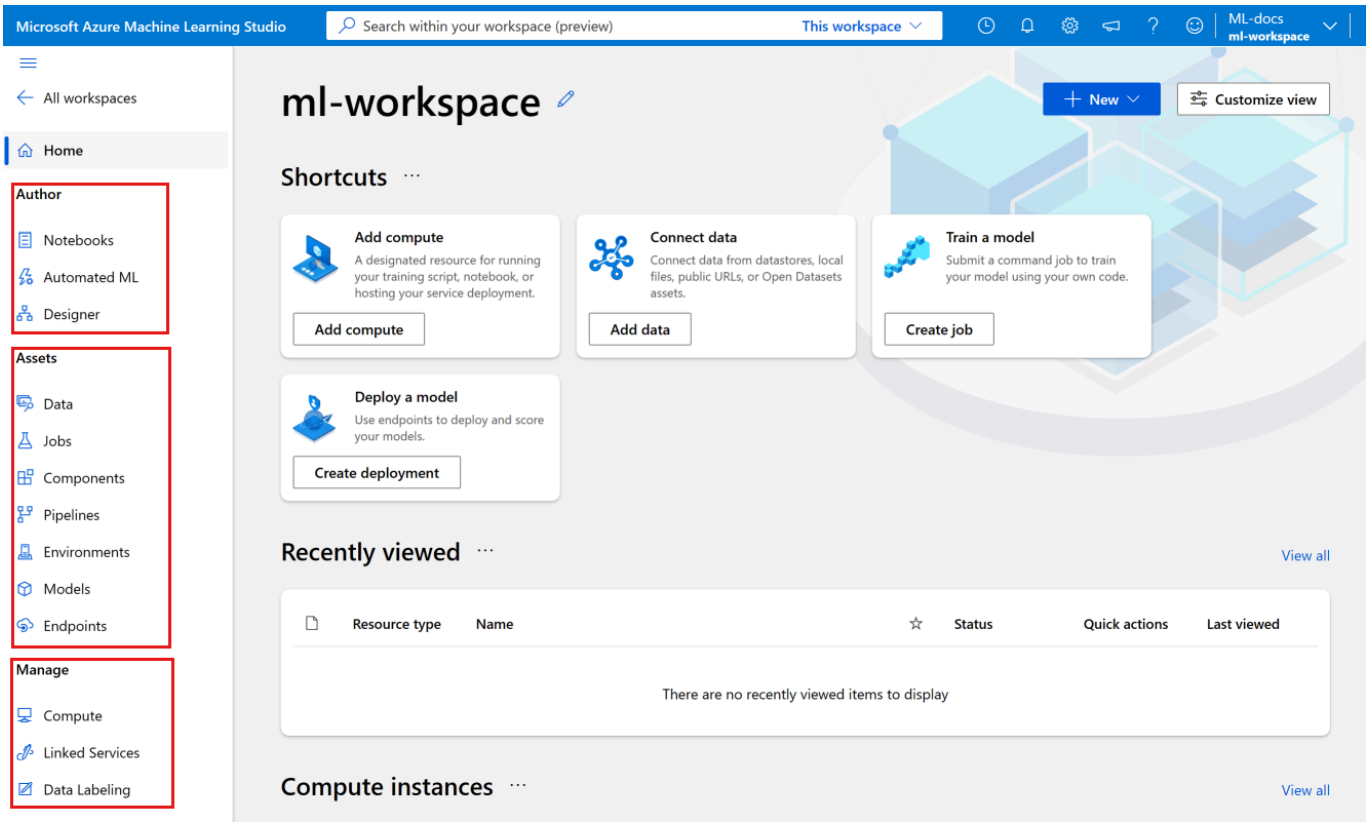
Azure ML 的定位是「企业级 MLOps 平台」：不只是「上传数据点一下开始训练」，而是把 **数据资产** → **环境** → **训练作业** → **模型注册** → **在线/批量推理** → **生产监控** 整条链路都做成了可版本化、可审计的资源对象。



（来源：Microsoft Learn，ML 开发闭环：定义任务 → 数据探索与准备 → 训练与验证 → 部署 → 监控与生命周期管理）

一、功能地图（控制台左侧导航）

Azure ML 的首页本身就是一张功能地图，左侧按 **Author / Assets / Manage** 三组划分，加上首页 4 个快捷入口：

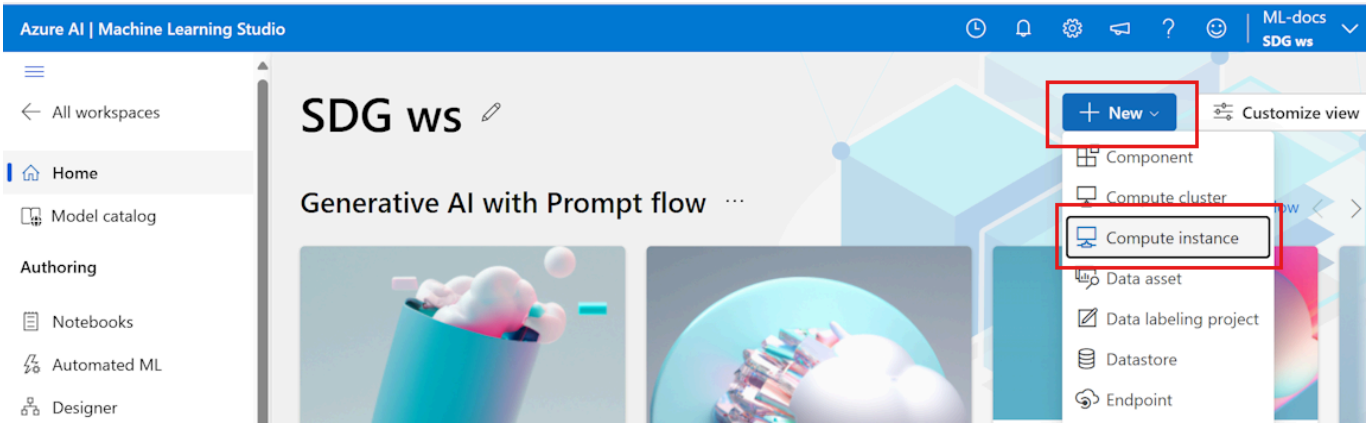


（来源：Microsoft Learn）

分组	导航项	对应课程核心需求
—	Home（Add compute / Connect data / Train a model / Deploy a model）	新手引导：4 步跑通全流程
Author	Notebooks	交互式开发（类似 Jupyter，与课程要求关系不大）
Author	Automated ML	**对标「预置模型调参」**：选数据 + 选任务类型，自动跑算法与超参搜索
Author	Designer	拖拽式无代码流水线（数据 → 组件 → 训练 → 部署）
Author	Model catalog	模型广场（几千个基础模型，可直接微调/部署）
Assets	**Data**	数据集管理（数据资产 + 版本 + 血缘）
Assets	**Jobs**	训练任务：提交、排队、状态、指标、日志、产物
Assets	Components	可复用流程组件（流水线的积木）
Assets	Pipelines	多步骤训练流水线
Assets	Environments	运行环境（Python 包 / Docker 镜像）
Assets	**Models**	模型产物管理（模型注册表 + 版本）
Assets	**Endpoints**	推理服务发布（在线端点 / 批量端点）

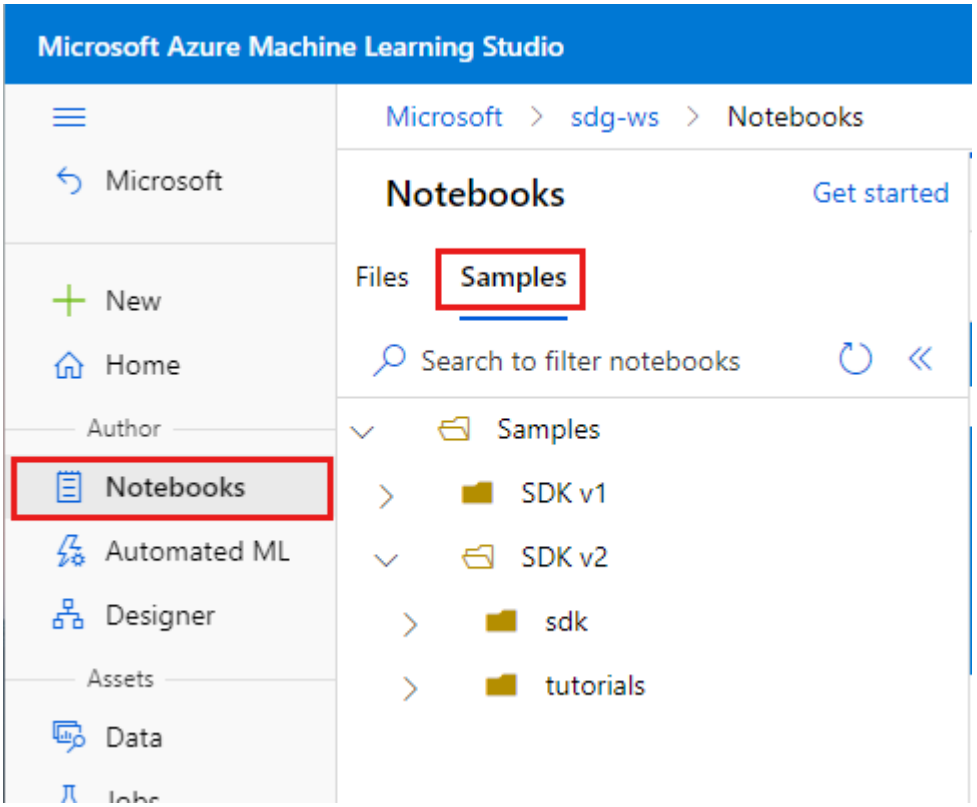
分组	导航项	对应课程核心需求
Manage	**Compute**	计算资源管理（实例 / 集群 / 无服务器 / 已接入集群）
Manage	Linked Services	外部连接（存储、AKS 等）
Manage	**Data Labeling**	智能标注项目（图像 / 文本）

「新建」菜单同样是一张功能地图，能一次看到所有可创建的资源类型：



（来源：Microsoft Learn，注意菜单里同时有 Compute cluster / Compute instance / Data asset / Data labeling project / Datastore / Endpoint / Component）

Azure 的「快速入门文档」不是一堆说明文字，而是内置在 Notebooks > Samples 里的可运行示例， 按 SDK v1 / SDK v2 / tutorials 分类，用户可以直接打开就跑：



（来源：Microsoft Learn）

二、完整流程（我们怎么做一次「训练 + 推理」）

Azure ML 的一条主线（对照「选择模型 → 申请资源 → 设置参数 → 启动 → 实时查看状态 → 获得结果」）：

```
工作区 Workspace
   └ 计算 Compute（Compute instance 开发机 / Compute cluster 训练集群 / Serverless
免建集群）
   └ 数据 Data（Datastore 存储连接 → Data asset 数据资产，带版本）
   └ 环境 Environment（预置 curated / 自定义 conda+docker）
      └ 作业 Job（command 单次训练 / sweep 超参搜索 / pipeline 多步流水线 / AutoML
自动调参）
         └ 运行监控（Metrics 曲线 / Logs 日志 / Child jobs / Outputs+logs）
            └ 模型 Model（模型注册表，带版本与标签）
               └ 端点 Endpoint → 部署 Deployment（在线推理 / 批量推理）
                  └ 生产监控（数据漂移 / 模型性能 / 触发重训）
```

1. 工作区（Workspace）

所有资源的顶层容器：作业历史（日志、指标、输出、代码快照）、数据、模型、环境、组件全部挂在一个工作区下。 管理方还可以用 **Registry（注册表）** 在多个工作区/团队之间共享模型和组件——这是「平台运营方」视角的重要模块。

2. 计算资源（Compute）

类型	用途	关键设计点
Compute instance	云端开发机（Notebook/VS Code）	单节点，按人分配，可停止计费
Compute cluster	训练集群（CPU/GPU）	**min/max 节点自动伸缩**、空闲自动缩容（`idle_time_before_scale_down`）、可建多节点通信型集群做分布式训练
Serverless compute	免建集群，提交作业时直接指定	**不需要运维集群，管理方最省事**；适合资源受限场景
Inference cluster	推理用 AKS 集群	自建或接入已有 K8s
Attached compute	接入外部算力	可把自有机器挂进来用

Azure 的三层管控：

1. **配额（Quota）**：按订阅 + 区域 + VM 系列限制核数。**GPU 系列的默认配额是 0 核**，必须先申请提额——和我们平台里「GPU 资源要先分配」是同一件事。
2. **集群上限**：`max_instances` 直接限制单个作业能占用多少节点；总计算资源上限默认 500/区域（可提到 2500），训练集群、开发机、在线部署共享这个额度。
3. **角色权限**：见下面第 9 节。

低成本技巧（可当创新点参考）：Low-priority / 抢占式节点，价格低但会被抢占，**作业最长 7 天**（普通作业 21 天），官方建议脚本里做好 checkpoint。

3. 数据接入（Data）

分成两层：

- **Datastore（数据存储）**：只保存「连接信息」（Blob 容器 / 文件共享 / ADLS Gen2），凭证不写进脚本。
- **Data asset（数据资产）**：指向数据的一个**不可变版本**，有三种类型：`uri_file`（单文件）、`uri_folder`（文件夹，图片/文本等非结构化）、`mltable`（表格，带 schema）。

数据资产解决了课程要求里「数据集管理」的几个真问题：

能力	说明
版本化 + 不可变	版本一旦创建不能改也不能删，因此**训练可复现**
血缘 Lineage	能看到哪个作业/流水线用了这个数据、数据又来自哪个作业输出
友好引用	用 `azureml:<名字>:<版本>` 代替长 URIs
消费方式	作业里可以 **mount（挂载）或 download（下载）** 到计算节点；大文件挂载更省时间
来源	本地路径（自动上传到默认存储）、Datastore、公开 https、Azure Storage
打标签 / 归档	Tag 用于分类；Archive 用于「下架但保留」，不能直接删被引用的版本

从作业输出直接生成新数据资产（`Creating data assets from job outputs`）是它做「数据流转」的关键：**作业产物 = 下一轮训练的数据源**。

4. 环境（Environment）

- **Curated 预置环境**：微软提供、开箱即用（常用框架）。
- **Custom 自定义环境**：Docker 镜像 / 基础镜像 + Conda YAML / Docker build context。

环境也是**版本化资产**，目的是让「训练环境可复现、可审计、可移植」。

5. 提交训练作业（Jobs）

这一块 Azure 提供了四条不同门槛的路径：

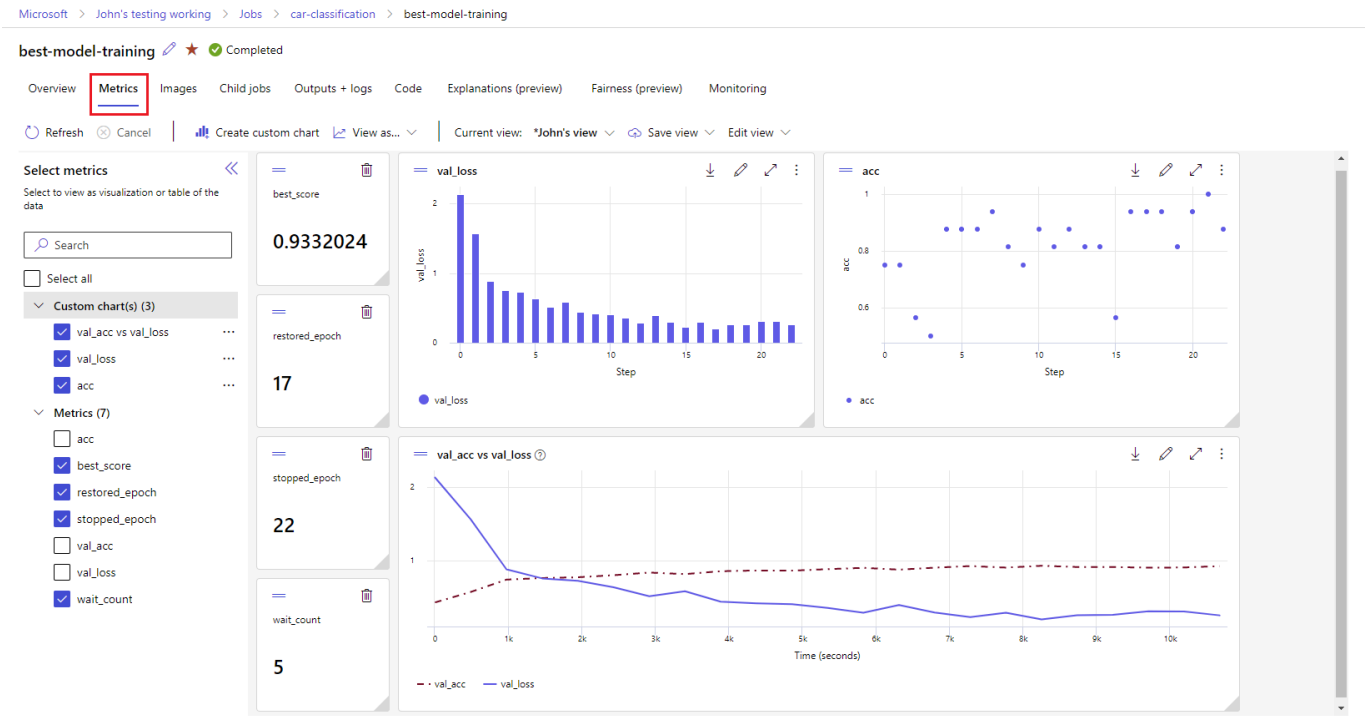
方式	适合	与课程要求的对应
Command job	一个脚本 + 参数，最基础的一次训练	核心：选数据/算力/超参 → 提交
Sweep job	超参搜索（自动调参，搜索空间 + 采样算法 + 早停策略）	创新点：自动超参搜索
Pipeline job	多步骤（数据处理 → 训练 → 评估 → 注册），步骤可复用	完整流程的工程化
Automated ML	完全不写代码：选数据和任务类型，自动试算法 + 调参 + 出可解释性报告	**对标百度 BML 的「预置模型调参」**

提交作业时要指定：数据资产版本 + 计算集群 + 环境 + 脚本 + 超参，也就是课程要求点 1 说的「选模型、选数据集、选资源、配超参」。 作业提交后有 instance_count（多机）、distribution（PyTorch/TensorFlow/MPI 分布式）等配置，分布式训练靠这个入口。

训练产物：作业输出（outputs）、日志（logs）、以及写回的模型，都会被工作区保存，可指定为新的数据资产或注册成模型。

6. 运行监控：状态 / 指标 / 日志 怎么看

这是最值得抄的前端设计。作业详情页顶部按标签页分成： Overview / Metrics / Images / Child jobs / Outputs + logs / Code / Explanations / Fairness / Monitoring。



（来源：Microsoft Learn，Metrics 页：左侧选指标，右侧自定义图表（折线/柱状）、可以「创建自定义图表」「保存视图」，上方有指标卡片如 best_score / restored_epoch / stopped_epoch / wait_count）

可以看到几条具体设计经验：

- **指标卡片 + 图表双区**：关键标量（best_score、最优 epoch）用大数字卡片，趋势用图表，一眼看完。
- **图表可自定义 + 可保存视图**：用户自己选指标组合、图表类型，并保存成「视图」反复看（多人协作时很有用）。
- **按次运行的 step/time 双轴**：横轴可以按 step 也可以按秒，方便定位「是哪个阶段开始变差的」。
- **Images 标签页**：直接看训练过程中记录的图片产物（CV 任务友好）。
- **Outputs + logs**：日志文件 + 诊断日志可下载；失败时会给出错误码与原因。
- 作业状态机大致是：NotStarted → Queued → Preparing → Running → Finalizing → Completed / Failed / Canceled，排队状态可见。

7. 模型产物管理（Models）

模型注册表：训练产物（本地文件 / 远程路径）注册成模型，带**名字 + 版本 + 标签 + 描述 + 属性**，底层支持 custom_model、mlflow_model、triton_model 三种格式（MLflow 格式可以免评分脚本直接部署）。

管理动作：列出 / 查看 / 更新（改标签、改描述）/ 归档。模型可作为**下一个训练作业的输入**（微调、增量训练），这是闭环的关键。

8. 推理服务发布（Endpoints）—— 课程要求「在线推理」的完整答案

Azure 把「端点（Endpoint）」和「部署（Deployment）」分开，这是整个设计里最漂亮的一处：

- **Endpoint**：稳定的调用 URL + 认证 + 授权 + 路由。
- **Deployment**：真正跑模型的资源 + 模型资产 + 环境 + 评分脚本。
- **一个端点下可以挂多个部署**，并支持 **流量切分（traffic split）** 和 **镜像流量（mirror）** 做灰度/蓝绿发布。

类型	场景	关键差异
Standard deployment	基础模型即开即用	无服务器、按 token 计费、不占订阅配额
Online endpoint	实时低延迟推理	需要配额；**按资源使用自动伸缩**；支持 Key + Entra 认证；有 Swagger；提供本地测试与在线测试面板
Batch endpoint	大批量离线打分	廉价：不调用不计费、可缩到 0、可用低优先级节点；**按作业数自动伸缩**；支持把「流水线」当作部署对象，模型和组件都能部署
Kubernetes online endpoint	自有 K8s	需要自建集群

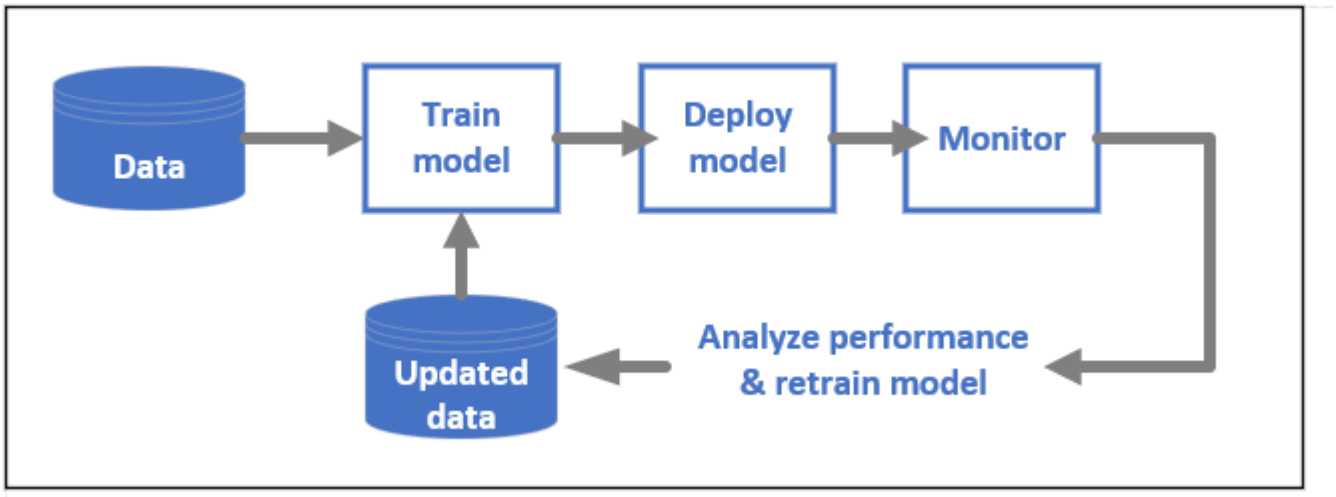
其他要点：

- 端点和部署有明确的配额/上限：每订阅每区域 100 个端点、500 个部署、每部署 50 实例、请求超时 180 秒、500 RPS 等。
- **在线端点会自动预留额外 20% 配额**用于系统升级（申请 10 个实例要有 12 个实例的配额）。
- 部署后可以拿到调用 URL + Key，也能在 Studio 里直接测；可以看端点指标、调用日志。

9. 生产监控与回流（Model monitoring）

这是 Azure 比国内几家平台做得更深的地方：把「模型上线后会不会变坏」也做成了平台功能。

- 在**在线端点**上开启 inference data collection（数据收集器），生产输入输出会被自动采集。
- 监控信号：**数据漂移、预测漂移、数据质量、特征归因漂移、模型性能**，还能自定义监控信号。
- 对比基线：训练数据或近期生产数据；可只看 Top N 重要特征。
- 每个信号可设阈值，超阈值后通过 Azure ML 或 Event Grid **触发告警**，并可以**触发自动重训**（重训后回流到 staging/test）。



（来源：Microsoft Learn，监控 → 分析性能 → 更新数据 → 重新训练的闭环）

10. 权限与角色（管理方需求）

用 Azure RBAC：先给 Microsoft Entra ID 里的用户/安全组分配角色，再作用到工作区。

内置角色	权限
AzureML Data Scientist	工作区内除「创建/删除计算」和「改工作区本身」之外的全部操作
AzureML Compute Operator	创建、管理、删除、访问计算资源（**就是课程要求里管 GPU 的那个角色**）
Reader	只读
Contributor	工作区内资产的增删改查 + 建集群 + 提交作业 + 部署
Owner	全部权限 + 改角色分配
AzureML Registry User	注册表内资产的读写删（不能创建/删除注册表本体）

经验点：

- **角色可以叠加**：一个人同时给 Data Scientist + Compute Operator，就能「自己做实验 + 自助建算力」，这是分布式算力场景常用的配置。
- **权限影响 UI**：没有建计算权限的用户，Studio 里干脆不显示「创建计算」入口，而不是等他点击后报 403。——这个细节我们的前端可以学。
- 支持 **自定义角色**（限定到订阅/资源组/工作区级别）、支持用 Entra 安全组批量管人。

三、对产品的 argue（我们保留什么、丢什么）

建议保留

- 「快速入门 / 4 步引导」首页（Add compute → Connect data → Train → Deploy）
- **模型广场（Model catalog）**：即使只是展示预置模型列表，也要有
- 「新建」统一入口 + 按 Author / Assets / Manage 分组的左侧导航
- 数据资产版本 + 血缘、模型注册表 + 版本（不需要做到 Azure 那么重，但概念要留）
- 端点/部署分离 + 流量切分（可以只做「在线推理 + 测试调用」的简化版）
- 作业详情页的标签页结构（Overview / Metrics / Logs / Outputs / Code）

- 「无权限就不显示入口」的权限联动

无需保留

- Azure 的市场/合作伙伴、订阅计费、成本分析、支持工单
- 多工作区 + 注册表 + 跨区域的复杂拓扑
- VS Code 远程、GitHub/Azure DevOps 集成、CI/CD 流水线
- 企业安全（VNet、CMK、私有端点）——可作为「背景介绍」一句话带过

需要改造的

- Azure ML v2 的文档和操作是**代码优先**的（CLI / Python SDK / YAML 是主路径，Studio 是壳）。我们课程要求的是**界面优先、预设调参为主**，所以 AutoML/Designer 那种「少写代码」的路径才是我们的参考对象。
- Azure 的「环境」概念对课程项目过重，建议简化为「预置环境模板 + 可改几个参数」。

四、每个需求大致的业务逻辑

课程需求	Azure 对应功能	业务逻辑（数据流）
选模型、数据集、算力，配超参，提交训练	Jobs：command / sweep / AutoML / pipeline	数据资产（版本）+ 环境 + 计算目标 + 超参 → 打包提交 → 进入队列
排队 / 调度	Compute cluster 自动伸缩 + 抢占式节点 + 配额	提交后集群按需扩容、空闲缩容到 0；超配额直接拒绝提交
展示状态、指标、日志	Job 详情页（Overview/Metrics/Images/Child jobs/Outputs+logs）	运行中持续写入 metrics 与 logs，前端轮询/订阅展示；指标可自定义图表并保存视图
保存训练产物	Job outputs → Data asset / Model registry	输出目录版本化，模型按名字+版本注册，可再作为输入
数据集管理	Datastore + Data asset（file/folder/mltable）	存储连接与数据资产解耦；版本不可变；血缘可查上下游
模型产物管理	Model registry	注册、版本、标签、归档；MLflow 格式可免评分脚本部署
推理服务发布	Online endpoint / Batch endpoint（+ 多部署 + 流量切分）	端点（URL + 认证 + 路由）与部署（模型 + 环境 + 算力 + 评分脚本）分离；灰度、自动伸缩、测试面板
用户权限、GPU 资源管理	Azure RBAC + Quota + 集群上限	角色决定「能做什么」，配额决定「能用多少」，两者独立
（加分）智能标注	Data labeling project	建标注项目 → 选数据集与类型 → 定义标签类别与说明 → 派工（标注工/团队）→ ML 辅助标注 → 增量刷新 → 质检
（加分）上线后监控	Model monitoring + data collector	采集生产输入输出 → 与基线做统计对比 → 阈值告警 → 触发重训回流

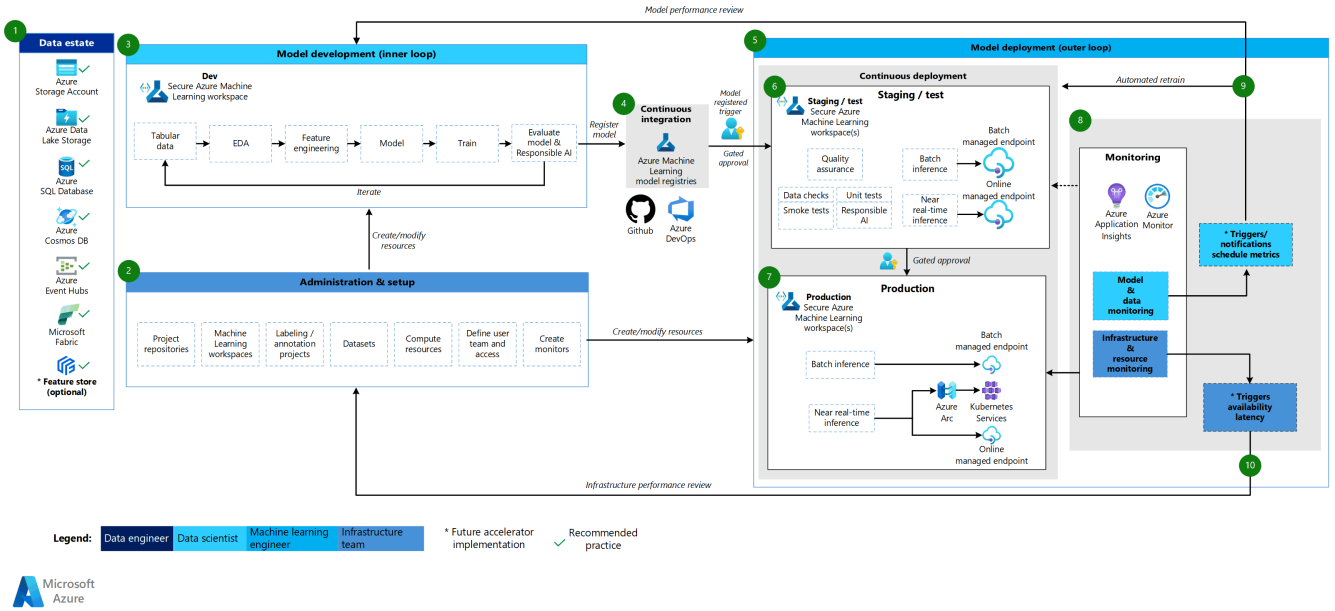
五、我们没想出怎么设计的点 → Azure 的做法

我们的疑问	Azure 的实际做法	我们可以怎么用
数据集上传后怎么管	Datastore 只存连接，Data asset 存「不可变版本」；file / folder / table 三种类型决定后续怎么读	简化版：数据集 = 名字 + 版本 + 类型（表格/图片/文本）+ 来源；版本不可改，只可新建版本
数据预处理到底是什么	一是 MLTable 里的转换（如支持 CSV 多行字段），二是把预处理写成一个 pipeline 组件，产物作为新数据资产	我们可只在数据集详情里提供「清洗/转换/切分训练测试集」几个固定操作，并把结果保存为新版本
日志和失败怎么展示	作业状态机 + 「Outputs + logs」标签页 + 诊断日志下载 + 失败时给错误码与原因	我们做：状态徽章 + 实时日志流 + 失败面板（错误码 + 原因 + 「一键重试」）
结果和图表怎么呈现	指标卡片（best_score 等）+ 可自定义图表 + 可保存视图 + Images 标签页 + Explanations/Fairness 报告	我们做：训练结果页 = 关键指标卡片 + Loss/Acc 曲线 + 子任务列表 + 产物下载
分布式训练怎么做	作业里设 `instance_count` + `distribution`（PyTorch/TF/MPI），计算集群建多节点通信型集群	我们可设计「节点数」输入项，把分布式细节留给后台（课程项目可以只做入口不真跑）
断点续训怎么做	**平台没有「一键续训」**：由脚本定期写 checkpoint 到作业输出，重训时把 checkpoint 作为输入；抢占式节点作业最长 7 天，官方明确提示要自己写 checkpoint。流水线层面有步骤级复用/失败继续	我们的「从检查点恢复」应该定位成：训练脚本约定 + 重提作业时选 checkpoint 文件，而不是平台魔法
在线推理怎么落地	Endpoint / Deployment 分离，多部署 + 流量切分 + 镜像流量；认证用 Key 或 Entra；有本地与在线测试	我们简化成：Endpoint（URL + Key）→ 1..N 个 Deployment（模型 + 环境 + 算力）→ 测试面板 + 调用记录
用户端权限要做到多少	6 个内置角色 + 自定义角色 + Entra 组；无权限的功能入口直接隐藏	我们做 3 个角色足够：管理员（管用户/GPU/全局配额）、普通用户（管自己的数据/作业/模型，可发布推理）、只读访客
部署也要占 GPU 吗	要。在线端点按部署实例占配额，且要额外预留 20% 用于升级；基础模型走「标准部署」不占配额、按 token 计费	我们的「在线推理占用 GPU」是可以明确写进需求文档的事实；同时可以设计「共享/独占」开关作为创新点

六、可以借鉴 / 可以当创新点的部分

1. **Endpoint / Deployment 分离 + 流量切分**：国内几家（含百度 BML）在推理这块通常只是「一个模型 = 一个服务」。Azure 的多部署 + 灰度这一层可以当我们的创新点，而且实现成本不高（多个部署后端 + 权重路由）。

2. **数据资产不可变版本 + 血缘图**：直接解决「数据集管理混乱」这个问卷里排名很高的痛点。
3. **Serverless compute（免建集群）+ 共享配额池**：对资源受限场景非常友好。共享配额池（写清「仅供短期测试」）这个思路，配合腾讯云 GPU 虚拟化，可以做我们平台的算力调度创新点。
4. **抢占式/低优先级节点**：用便宜算力换「可能被抢占」，逼着训练脚本写 checkpoint——很适合我们「对接远端笔记本 GPU / 实验室 GPU 等不稳定算力」的背景。
5. **指标视图可保存**：个人/团队各自保存一套「我最关心的图表组合」。
6. **权限与配额分离**：「你这个角色能提交作业」和「你能用多少 GPU」是两件事，管理后台要分成两个页面。
7. **上线后监控 + 事件触发重训**：可作为「创新功能」章节的压轴，即使只做到「调用记录 + 简单漂移统计」。
8. **AutoML 的呈现方式**：自动跑多个尝试（child jobs），结果用排行榜对比，挑最好的一个再部署——这正是「一个项目下多个训练任务可对比」的现成参考。



（来源：Microsoft Learn / Azure Architecture Center，MLOps v2 经典机器学习架构：左边是数据资产与管理配置，中间是模型开发内循环，右边是持续部署、生产、监控与回流）

七、核心需求梳理（简短版）

Azure ML 调研下来，用户的核心需求可以归成 6 条：

1. **能跑通一次训练**：选数据 → 选算力 → 配参数 → 提交 → 看进度 → 拿结果。（这是最低门槛，任何设计不能让它变复杂）
2. **数据和模型要「管得住」**：有版本、能追溯、能复用、能找人。
3. **资源要「管得住」**：谁能用、能用多少、用完怎么释放。管理方需要配额 + 权限两把锁。
4. **训练过程要「看得见」**：状态、进度、指标曲线、日志、失败原因，缺一个用户就要去问人。
5. **成果要「能交付」**：模型能一键变成可调用的推理服务，最好还能灰度、能看调用情况。

6. **出问题要「能闭环」**：失败能重试/续训，上线能监控，变差能重训。

注：Azure 账号受限，调研基于官方文档整理。